

CMP501: Network Game Development – Technical Report

Project: Networked Tank Shooter (UDP Implementation)

Kananake Yimsirivattana 2502432

1. Introduction

This report details the technical implementation and critical evaluation of a real-time networked tank game. The primary objective was to transition from a TCP-based model to a high-performance UDP architecture. The report focuses on how the system manages the inherent unpredictability of network conditions through synchronization techniques and evaluates its performance under both ideal and simulated lag conditions.

2. Network Architecture and Protocol (LO1)

2.1 UDP Selection

For a fast-paced action game, minimizing latency is more critical than guaranteed delivery. Following the industry-standard principles (Fiedler, n.d.), **UDP** was selected as the transport protocol. Unlike TCP, UDP avoids "Head-of-Line Blocking," ensuring that the game state remains current even if some packets are dropped, which is essential for maintaining the "flow" of a real-time shooter.

2.2 Server-Relay Topology

The project utilizes a Server-Relay architecture. The Server acts as a central authority that receives state updates from each client and broadcasts them to all other participants. This topology ensures that the Server maintains the most accurate "world state" while reducing the bandwidth burden on individual clients compared to a Peer-to-Peer model. In Figure 1, They have three screens. Role1 is Client 1, Role 2 is Client 2 and Role 3 is Server.

Red tank is player 1 and green tank is player 2. Figure 2 When Client moves and shoots. Server and other client receive data.

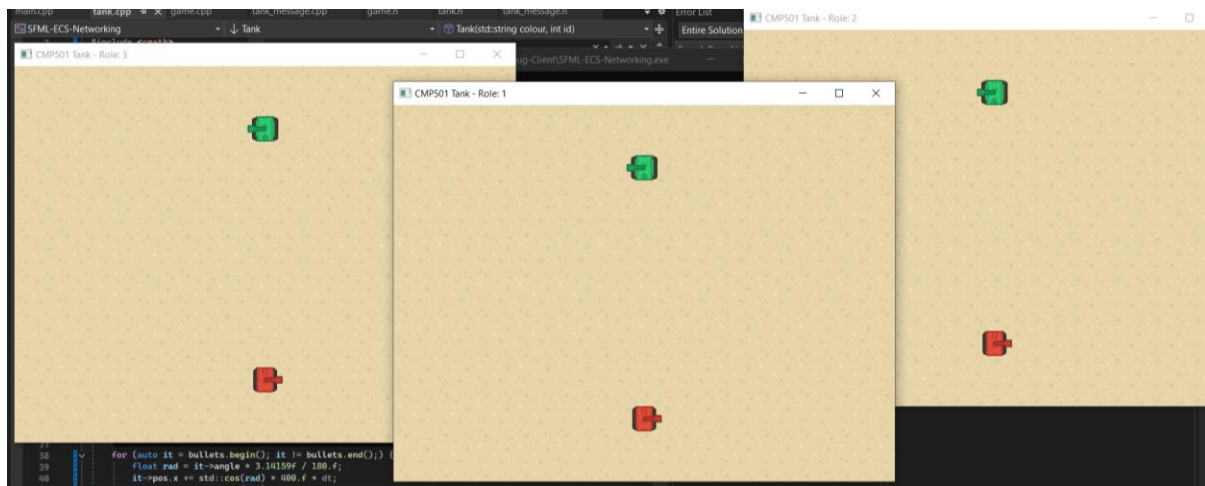


Figure 1 Three screens

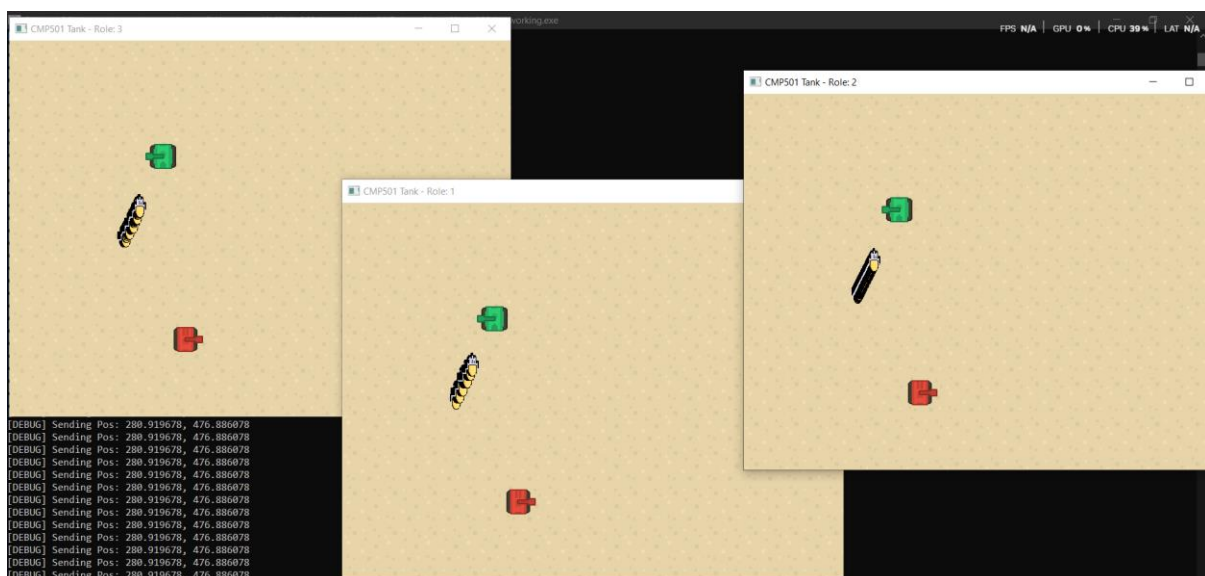


Figure 2 Interaction data

3. Mitigation of Network Conditions (LO2)

3.1 Entity Interpolation and Design Intent

To combat network jitter, an **Entity Interpolation** system was implemented. Remote tank data is stored in a positionBuffer and rendered with a defined interpolationDelay of 100ms.

- **Technical Reason for Delay:** This deliberate delay ensures the client always has a "window" of historical data to interpolate between, creating smooth movement instead of "snapping" to the latest raw packet.

3.2 Local Prediction

The local player's tank uses **Local Authoritative Movement**. Input is processed immediately on the local client to provide a latency-free experience for the user, while the resulting state is simultaneously dispatched to the Server.

4. Testing and Critical Evaluation (LO3)

4.1 Baseline Performance (Ideal Conditions)

For differentiate between the two networked entities, the system maps **WASD** to the Red Tank (Player 1) and **Arrow Keys** to the Green Tank (Player 2). This mapping is handled within the Tank::Update and Game::InitLocalTank functions. The use of local input handling allows for **Immediate Feedback**, a core principle of high-performance game networking. While the local player moves according to their specific key bindings, the resulting coordinates are packaged into a TankMessage and dispatched via UDP, ensuring that the control scheme does not interfere with the underlying synchronization logic.

During initial testing without artificial latency (No Clumsy), a distinct but intentional behaviour was observed:

- Local Client: Movement was instantaneous and perfectly smooth.
- Remote Clients: A slight, constant delay was visible when observing other players.
- Analysis: This confirms that the Interpolation Buffer is functioning correctly. The observed delay on remote screens is the sum of the network propagation time and the 100ms buffer delay. This is a deliberate trade-off: we sacrifice absolute synchronicity to gain visual stability and smoothness across all clients.

4.2 Stress Testing with Clumsy

The system was further evaluated using **Clumsy 0.3** to simulate degraded network environments:

- Latency Simulation (100ms+): As artificial lag was introduced, the delay between the Server's view and the Clients' views increased. However, thanks to the interpolation logic, the remote tanks continued to move fluidly without "teleporting" or jittering, proving the robustness of the buffer.
- Inter-Client Interaction: It was observed the delay between two clients (Client A seeing Client B) was slightly higher than the Server's delay. This accurately reflects the Server-Relay path (Client -> Server -> Client), confirming the architectural behaviour.

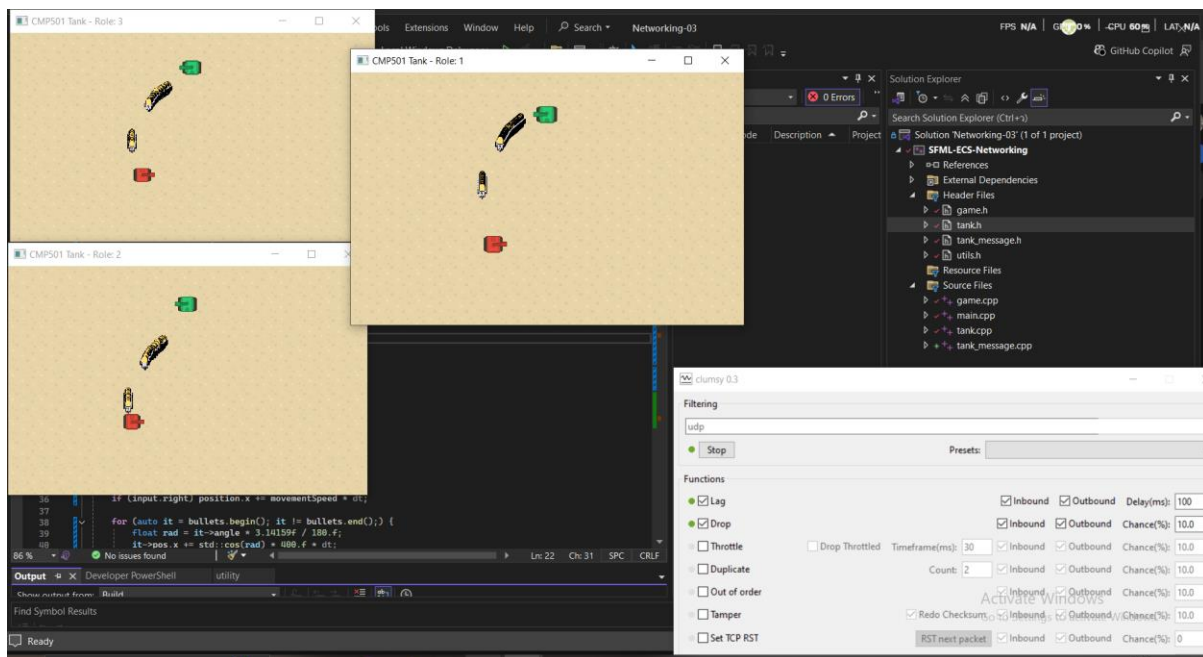


Figure 3 Testing with Clumsy 0.3

5. Conclusion

The implementation successfully meets the learning outcomes of the module. The transition to UDP provided the necessary speed for a fast-action game, while the interpolation system effectively masked network inconsistencies. The testing phase verified that the deliberate delay in synchronization is a vital component in maintaining a smooth and playable experience under varying network conditions.

6. References

1. Fiedler, G. (n.d.). UDP vs TCP. Available at: https://gafferongames.com/post/udp_vs_tcp/
2. Valve Developer Community. (n.d.). Source Multiplayer Networking. Available at: https://developer.valvesoftware.com/wiki/Source_Multiplayer_Networking
3. SFML Documentation. (2024). Network module - Communication with packets. Available at: <https://www.sfml-dev.org/tutorials/2.6/network-packet.php>
4. Kawick, M. (1999). Real-Time Strategy Game Programming using MS DirectX 6.0. Wordware Publishing.